

PROJECT REPORT

Optimizing Emergency Response with Fire Incident Data - A Case Study of Boston

❖ Introduction

Emergency services, especially fire departments, are vital for safeguarding lives and property during crisis events. However, challenges such as delayed responses and misallocation of resources can hinder effective firefighting. With the increasing availability of urban data, this project explores how Boston fire incident data can be used to predict high-loss fire events and assist fire departments in planning and prioritizing their emergency responses.

❖ Problem Statement

The core problem addressed is minimizing property damage and improving emergency response times using predictive analytics. By analyzing historical fire incident data, this project aims to:

- Enable quicker identification of high-risk fire incidents
- Reduce false alarms and resource wastage

This approach assists fire departments with proactive resource planning and operational efficiency improvements.

❖ Data Collection and Preprocessing

➤ Data Source

The dataset was sourced from the [Boston Fire Department's Open Data Portal](#)

It contains 38164 records and 24 columns, each representing a unique incident report from the year 2012. The key features include:

- Alarm Date and Time
- Incident Type and Description
- Estimated Property and Content Loss
- Location details: Zip code, District, Property Use
- Street and address-related metadata

➤ Data Cleaning

The following cleaning operations were performed:

- Dropped columns with excessive missing or irrelevant data (like Neighborhood, Address 2, and xStreet columns).
- Filled missing values in Estimated Property Loss and Estimated Content Loss using median values.
- Parsed date and time fields into standard datetime formats.

➤ **Feature Engineering**

To enhance the model's performance and provide context:

- Combined Estimated Property Loss and Estimated Content Loss into a new field: Total Loss.
- Created a binary target variable High_Loss_Fire, labeled 1 if Total Loss > \$300, else 0.
- Extracted time-based features:
 - Month
 - Day of the week
 - ISWeekend (binary:1 for Saturday/Sunday)
- Computed incident frequency by zip code and added it as a new feature (IncidentCount), reflecting fire-prone areas.

❖ **Model Implementation and Evaluation**

➤ **Feature Selection**

To determine the most relevant predictions, we applied:

- Mutual Information: to identify the non-linear dependencies
- Random Forest Feature Importance: to evaluate relative influence on classification.

➤ **Top predictive features**

- Incident Type
- Property Use
- Zip
- Dayofweek

These were used as inputs for modeling.

❖ **Model Building**

We implemented and compared following machine learning models:

❖ **Logistic Regression**

It is a statistical model that predicts the probability of a binary outcome (like high-loss vs low-loss fire). It assumes a linear relationship between the input features and the log-odds of the target class.

We used it because:

- Acts as a baseline classifier
- Fast and interpretable
- Useful to compare performance against more complex models.

Sample Code Implementation

```
# Logistic Regression
log_param_grid = {'C': [0.01, 0.1, 1, 10], 'penalty': ['l1', 'l2'], 'solver': ['liblinear']}
log_grid = GridSearchCV(LogisticRegression(class_weight='balanced'), log_param_grid, cv=5, scoring='f1')
log_grid.fit(X_train_scaled, y_train)
models['Logistic Regression'] = log_grid.best_estimator_
probs['Logistic Regression'] = models['Logistic Regression'].predict_proba(X_val_scaled)[: , 1]
predictions['Logistic Regression'] = models['Logistic Regression'].predict(X_val_scaled)
confusion_matrices['Logistic Regression'] = confusion_matrix(y_val, predictions['Logistic Regression'])
performance.append({
    'Model': 'Logistic Regression',
    'Accuracy': accuracy_score(y_val, predictions['Logistic Regression']),
    'Precision': precision_score(y_val, predictions['Logistic Regression']),
    'Recall': recall_score(y_val, predictions['Logistic Regression']),
    'F1-Score': f1_score(y_val, predictions['Logistic Regression']),
    'ROC-AUC': roc_auc_score(y_val, probs['Logistic Regression']),
    'CV_F1_Score': log_grid.best_score_
})
```

❖ **Random Forest Classifier**

It is an ensemble model that builds multiple decision trees on different subsets of the data and combines them to improve accuracy and reduce overfitting.

We used it because:

- Robust to overfitting
- Captures non-linear patterns and feature interactions.
- Provides feature importance scores.

Sample Code Implementation:

```
# Random Forest
rf_param_grid = {'n_estimators': [50, 100], 'max_depth': [5, 10], 'min_samples_split': [2, 5]}
rf_grid = GridSearchCV(RandomForestClassifier(class_weight='balanced'), rf_param_grid, cv=5, scoring='f1')
rf_grid.fit(X_train_scaled, y_train)
models['Random Forest'] = rf_grid.best_estimator_
probs['Random Forest'] = models['Random Forest'].predict_proba(X_val_scaled)[: , 1]
predictions['Random Forest'] = models['Random Forest'].predict(X_val_scaled)
confusion_matrices['Random Forest'] = confusion_matrix(y_val, predictions['Random Forest'])
performance.append({
    'Model': 'Random Forest',
    'Accuracy': accuracy_score(y_val, predictions['Random Forest']),
    'Precision': precision_score(y_val, predictions['Random Forest']),
    'Recall': recall_score(y_val, predictions['Random Forest']),
    'F1-Score': f1_score(y_val, predictions['Random Forest']),
    'ROC-AUC': roc_auc_score(y_val, probs['Random Forest']),
    'CV_F1_Score': rf_grid.best_score_
})
```

❖ **LightGBM Classifier**

A gradient boosting framework that uses tree-based learning algorithms, known for speed and efficiency.

We used it because:

- High efficiency and lower memory usage

- Supports parallel and GPU learning.
- Fast than other boosting algorithms.

Sample Code Implementation:

```
# LightGBM
lgb_param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [3, 5],
    'learning_rate': [0.01, 0.1],
    'num_leaves': [31, 50]
}
scale_pos_weight = sum(y_train == 0) / sum(y_train == 1)
lgb_grid = GridSearchCV(lgb.LGBMClassifier(scale_pos_weight=scale_pos_weight), lgb_param_grid, cv=5, scoring='f1')
lgb_grid.fit(X_train_scaled, y_train)
models['LightGBM'] = lgb_grid.best_estimator_
probs['LightGBM'] = models['LightGBM'].predict_proba(X_val_scaled)[:, 1]
predictions['LightGBM'] = models['LightGBM'].predict(X_val_scaled)
confusion_matrices['LightGBM'] = confusion_matrix(y_val, predictions['LightGBM'])
performance.append({
    'Model': 'LightGBM',
    'Accuracy': accuracy_score(y_val, predictions['LightGBM']),
    'Precision': precision_score(y_val, predictions['LightGBM']),
    'Recall': recall_score(y_val, predictions['LightGBM']),
    'F1-Score': f1_score(y_val, predictions['LightGBM']),
    'ROC-AUC': roc_auc_score(y_val, probs['LightGBM']),
    'CV_F1_Score': lgb_grid.best_score_
})
```

❖ XGBoost Classifier

It is a high-performance implementation of gradient-boosted trees. It builds trees sequentially, where each new tree attempts to correct errors made by the previous one.

We used it because:

- Extremely fast, accurate, and efficient.
- Handles imbalanced data better with built-in regularization.
- Widely used in real-world competitions (e.g., Kaggle).

Sample Code Implementation:

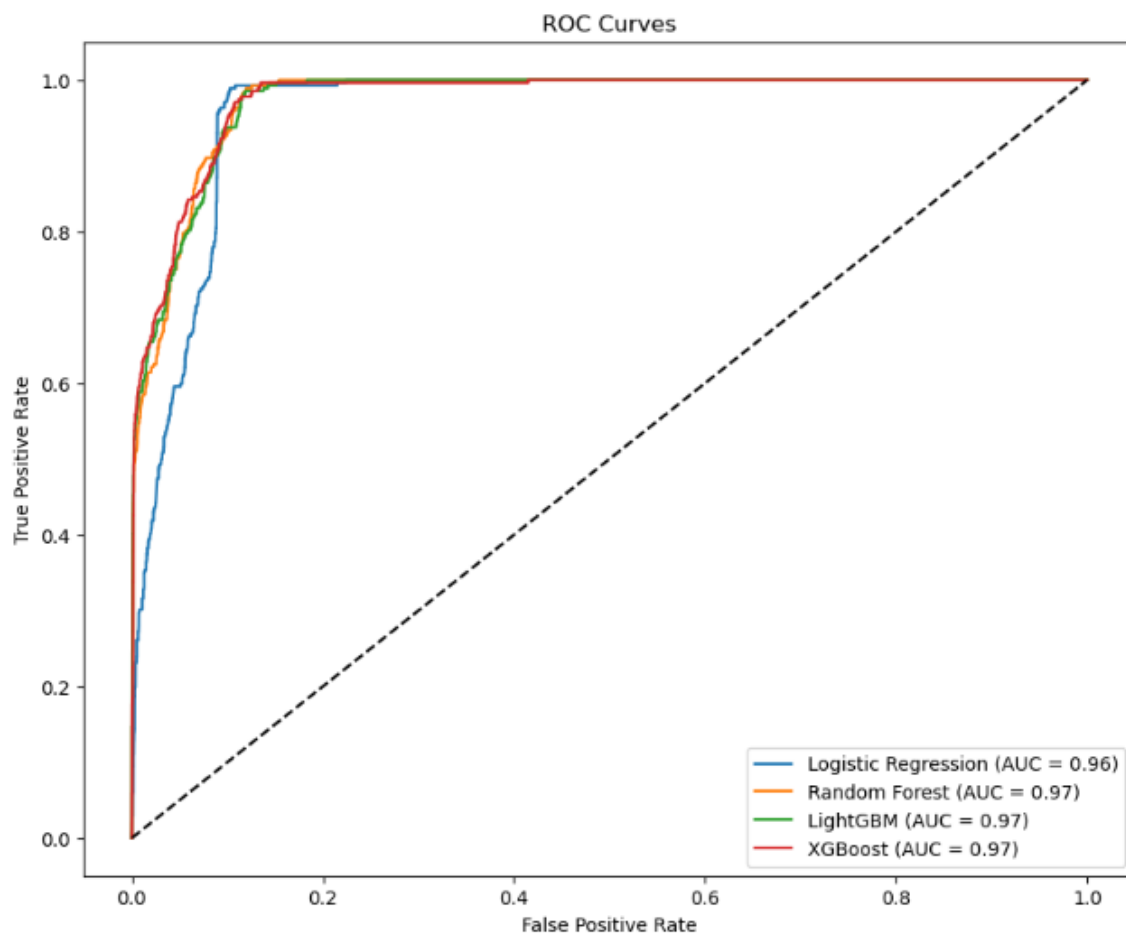
```
# XGBoost
xgb_param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [3, 5],
    'learning_rate': [0.01, 0.1],
    'scale_pos_weight': [sum(y_train == 0) / sum(y_train == 1)]
}
xgb_grid = GridSearchCV(xgb.XGBClassifier(), xgb_param_grid, cv=5, scoring='f1')
xgb_grid.fit(X_train_scaled, y_train)
models['XGBoost'] = xgb_grid.best_estimator_
probs['XGBoost'] = models['XGBoost'].predict_proba(X_val_scaled)[:, 1]
predictions['XGBoost'] = models['XGBoost'].predict(X_val_scaled)
confusion_matrices['XGBoost'] = confusion_matrix(y_val, predictions['XGBoost'])
performance.append({
    'Model': 'XGBoost',
    'Accuracy': accuracy_score(y_val, predictions['XGBoost']),
    'Precision': precision_score(y_val, predictions['XGBoost']),
    'Recall': recall_score(y_val, predictions['XGBoost']),
    'F1-Score': f1_score(y_val, predictions['XGBoost']),
    'ROC-AUC': roc_auc_score(y_val, probs['XGBoost']),
    'CV_F1_Score': xgb_grid.best_score_
})
```

❖ Evaluation Metrics

To assess model performance, we used the following metrics:

- Accuracy: Overall correctness.
- Precision: Correct positive predictions out of all positive predictions.

- Recall: Correct positive predictions out of all actual positives.
- F1 Score: Harmonic mean of precision and Recall.
- Roc-Auc Score: Measure how well the model separates the classes.

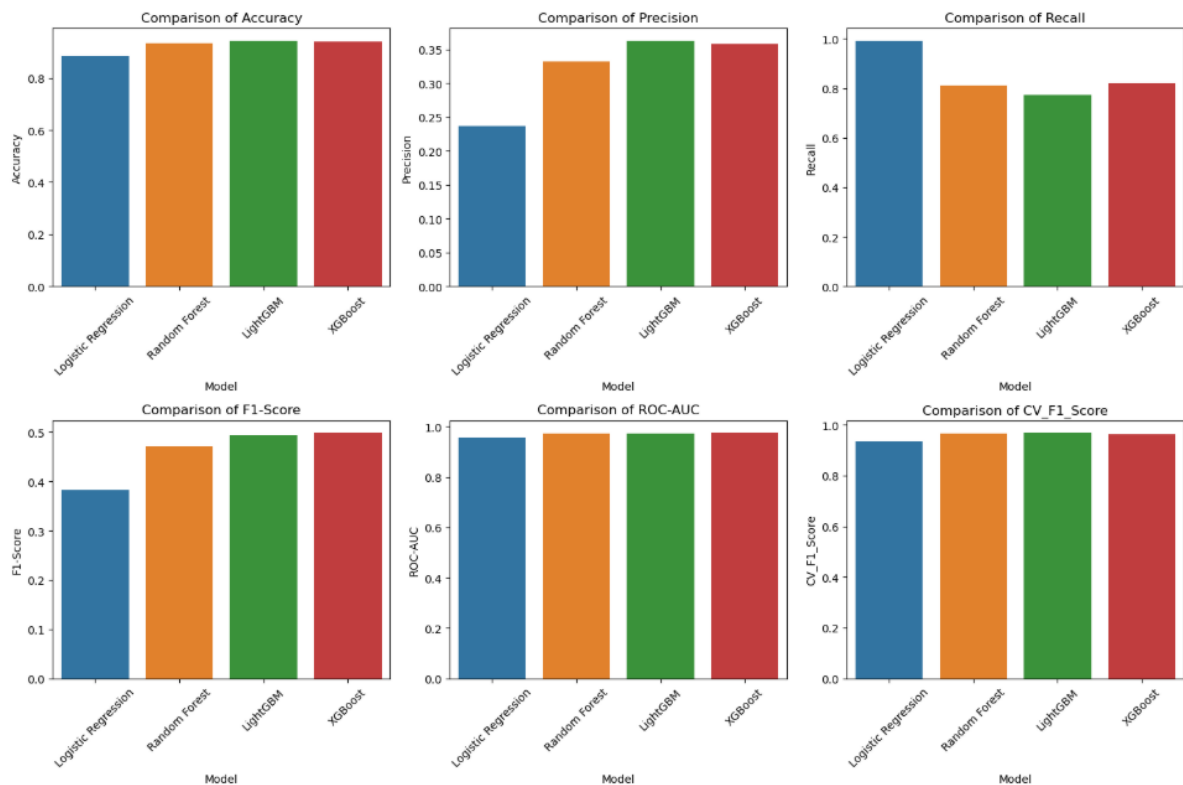


❖ Results

➤ Model Results and Performance Metrics

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV_F1_Score
Logistic Regression	0.618	0.055	0.603	0.101	0.759	0.613
Random Forest	0.906	0.251	0.827	0.385	0.965	0.949
LightGBM	0.921	0.287	0.824	0.425	0.967	0.954
XGBoost	0.911	0.274	0.901	0.42	0.969	0.946

➤ Comparison of Model Results



❖ Model Comparison Analysis

- LightGBM achieved the highest accuracy (0.921), closely followed by XGBoost and Random Forest.
- All tree-based models outperformed Logistic Regression, especially in recall and F1-score.
- Precision remains relatively low for all models, indicating a higher rate of false positives, but recall is strong-critical for emergency response prioritization.

❖ Findings and Interpretations

- **Fire Loss Severity** - Most incidents are low-loss, but high-loss events, though rare, are crucial for resource allocation.
- **Seasonal Trends** - Higher incident rates in winter months (January–March), likely due to heating-related fires.
- **Geographical Risk Zones** - Certain zip codes are consistently linked to higher property losses and should be prioritized.

- **Visual Insights** - Bar plots, line graphs, and zip code charts help stakeholders understand when and where interventions are most needed.

❖ **Conclusions and Recommendations**

This study demonstrates the power of data-driven methods in enhancing public safety through emergency response planning. By applying machine learning to historical fire incident data, we identified key patterns and developed high-performing predictive models.

❖ **Key Findings:**

- LightGBM is the most accurate model for predicting high-loss fire events, with XGBoost and Random Forest also performing strongly.
- Time and location features are strong predictors.
- Fire-prone zones and seasons can be targeted for preventive campaigns and resource distribution.

❖ **Recommendations:**

- Use predictive models to prioritize dispatch during peak months or in high-risk zones.
- Launch seasonal safety campaigns, especially in winter.
- Invest in improved data tracking systems to further enhance incident reporting and model refinement.

❖ **References**

1. Boston Fire Department. Fire Incident Reporting Dataset.
<https://data.boston.gov/dataset/fire-incident-reporting>
2. Python Libraries: pandas, matplotlib, seaborn, sklearn, xgboost, lightgbm.

❖ **Future Work**

Future research should incorporate real-time weather data (e.g., wind speed, temperature) to further enhance fire risk prediction. Continuous model updates with new incident reports will help maintain predictive accuracy over time.